

SportRAG

Football Analytics Retrieval-Augmented Generation System

Z2004: Database Management Systems
IIT Madras Zanzibar — Even Semester 2026
Track A: RAG Pipeline

Submitted by:

Simeon Mudenda — ZDA24B003
Patrick Muchindu — ZDA24B011
Sarim Rouf — ZDA23B028

Supervisor: Dr. Innocent Nyalala
June 2026

1. Project Overview

SportRAG is a football analytics system that combines a normalised PostgreSQL relational database with a vector similarity search layer to answer natural-language questions about match data. The system demonstrates database design, third normal form normalisation, SQL querying, index optimisation, and a fully functional retrieval-augmented generation (RAG) pipeline as required by Track A.

The database is hosted on Supabase (PostgreSQL 15) and contains data across five tables: Teams, Players, Matches, PlayerStats, and Documents. The RAG layer uses the all-MiniLM-L6-v2 sentence-transformers model to embed 1,000 match report documents into pgvector, enabling semantic similarity search. A FastAPI backend exposes the pipeline via HTTP endpoints, and a browser-based frontend provides a chat interface for end users.

1.1 Objectives

- Design and implement a normalised relational database schema (3NF)
- Generate and load a realistic football dataset of over 12,000 rows
- Implement a suite of SQL queries covering aggregation, joins, subqueries, CTEs, and window functions
- Optimise query performance using B-Tree indexes with EXPLAIN ANALYZE evidence
- Implement a stored procedure and trigger for automatic rating calculation
- Build a Track A RAG pipeline: local embeddings, pgvector storage, and grounded answer generation
- Deliver a working Python application with a browser-based frontend

2. Database Schema and Normalisation

2.1 Entity-Relationship Overview

The schema contains five entities with the following relationships:

- Teams 1 — * Players (one team has many players)
- Players 1 — * PlayerStats (one player has many stat records)
- Matches 1 — * PlayerStats (one match has many stat records)
- Matches 1 — * Documents (one match has many associated reports)

2.2 Table Definitions

Table	Columns	Primary Key	Foreign Keys
Teams	team_id, name, league	team_id	—
Players	player_id, name, position, team_id	player_id	team_id → Teams
Matches	match_id, date, home_team_id, away_team_id, home_score, away_score	match_id	home_team_id, away_team_id → Teams
PlayerStats	stat_id, player_id, match_id, goals, assists, minutes_played, rating	stat_id	player_id → Players, match_id → Matches
Documents	doc_id, match_id, content, source, embedding	doc_id	match_id → Matches

2.3 Normalisation to 3NF

The schema satisfies all three normal forms:

First Normal Form (1NF): All attributes contain atomic values. There are no repeating groups or multi-valued attributes. Each table has a well-defined primary key.

Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. In PlayerStats, goals, assists, minutes_played, and rating all depend on the composite (player_id, match_id) context identified by stat_id. No partial dependencies exist.

Third Normal Form (3NF): No transitive dependencies exist. Player position and team membership are stored in the Players table, not duplicated in PlayerStats. Team league information is stored only in Teams, not repeated in Matches or Players.

3. Dataset

3.1 Data Summary

Table	Row Count	Generation Method
Teams	20	Faker — realistic club names and leagues
Players	500	Faker — names, positions distributed realistically
Matches	1,000	Random fixtures between teams with realistic scores
PlayerStats	12,009	Per-match stats with rating auto-calculated by trigger
Documents	1,001	Match reports generated with domain-relevant text

3.2 Data Generation

All data was generated using a Python script (`generate_data.py`) using the Faker library and the psycopg2 database adapter. The generation process enforces referential integrity: players are assigned to existing teams, match stats reference existing players and matches, and documents reference existing matches. Scores follow a realistic distribution (0–5 goals per team). The dataset satisfies the Track A minimum of 1,000 rows of meaningful data.

4. SQL Query Implementation

Ten SQL queries were implemented covering the full range of required query types. All queries return correct, meaningful results verified against the live database.

Query	Description	Type
Q1	Top goal scorers across all matches	Aggregation + Join
Q2	Top assist providers	Aggregation + Join
Q3	Matches with the highest total goals	Aggregation
Q4	Players with above-average goals	Subquery
Q5	Teams with more than 10 total goals	Subquery + Join
Q6	Player goal rankings	Window Function (RANK OVER)
Q7	Running total of match goals	Window Function (SUM OVER)
Q8	Average goals per team	CTE
Q9	Most active players by minutes played	CTE + Aggregation
Q10	Match reports containing keywords	Text Search (ILIKE)

4.1 Sample Query: Top Goal Scorers (Q1)

```
SELECT p.name, SUM(ps.goals) AS total_goals
FROM PlayerStats ps
JOIN Players p ON ps.player_id = p.player_id
GROUP BY p.name
ORDER BY total_goals DESC
LIMIT 10;
```

4.2 Sample Query: Player Goal Rankings (Q6)

```
SELECT p.name,
       SUM(ps.goals) AS total_goals,
       RANK() OVER (ORDER BY SUM(ps.goals) DESC) AS goal_rank
FROM PlayerStats ps
JOIN Players p ON ps.player_id = p.player_id
GROUP BY p.name
ORDER BY goal_rank;
```

5. Performance Optimisation

5.1 Test Environment

- Database engine: PostgreSQL 15 (Supabase cloud)
- Dataset size: 12,009 PlayerStats rows (largest table)
- Timing method: EXPLAIN ANALYZE (averaged over 3 runs)
- Indexes applied after baseline measurements

5.2 Indexes Applied

Index Name	Table	Column	Type	Justification
idx_playerstats_player	PlayerStats	player_id	B-Tree	Speeds up joins to Players table in all aggregation queries
idx_playerstats_match	PlayerStats	match_id	B-Tree	Speeds up joins to Matches table
idx_players_team	Players	team_id	B-Tree	Speeds up team-based aggregations
idx_documents_match	Documents	match_id	B-Tree	Speeds up report retrieval by match
idx_documents_embedding	Documents	embedding	IVFFlat	pgvector cosine similarity search for RAG

5.3 Before and After Results

Query	Before Index	After Index	Improvement
Q1: Top Goal Scorers	13.46 ms	12.04 ms	10%
Q5: Teams With >10 Goals	19.07 ms	11.03 ms	42%
Q6: Player Goal Rankings	7.78 ms	7.71 ms	1%

5.4 EXPLAIN ANALYZE Findings

Before indexing, EXPLAIN ANALYZE showed sequential scans (Seq Scan) on the PlayerStats table as the dominant cost, combined with Hash Join and Hash Aggregate operations. The PlayerStats table at 12,009 rows was the most expensive to scan repeatedly.

After applying idx_playerstats_player and idx_players_team, Query 5 (Teams With More Than 10 Goals) showed the largest improvement at 42%, as it benefits from two index hits in the same query. The planner switched from a sequential scan to an Index Scan on PlayerStats,

then used a Nested Loop join to Players. Query 6 showed minimal improvement because the Window Aggregate operation dominates the execution cost regardless of the index.

6. Stored Procedure and Trigger

6.1 Trigger: calculate_player_rating

A BEFORE INSERT trigger was implemented on the PlayerStats table to automatically calculate a player's match rating whenever a new stat record is inserted. This eliminates the need for application-level rating computation and ensures data integrity.

6.2 Rating Formula

$$\text{rating} = (\text{goals} * 2) + \text{assists} + (\text{minutes_played} / 90)$$

Goals are weighted at 2 points each to reflect their greater match impact. Assists contribute 1 point each. Minutes played contributes proportionally (a full 90-minute game adds 1 point).

6.3 Implementation

```
CREATE OR REPLACE FUNCTION calculate_player_rating()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.rating := (NEW.goals * 2) + NEW.assists
                + (NEW.minutes_played::numeric / 90);
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_calculate_rating
BEFORE INSERT ON PlayerStats
FOR EACH ROW
EXECUTE FUNCTION calculate_player_rating();
```

6.4 Debugging and Fix

During initial testing, the trigger was found to be running (confirmed via information_schema.triggers) but rating values remained NULL after insertion. The root cause was identified: the trigger function was missing the RETURN NEW statement. In PostgreSQL, a BEFORE ROW trigger must return the modified NEW record for the changes to be written to the database. Returning NULL silently cancels the write. Adding RETURN NEW resolved the issue immediately. Existing NULL ratings were backfilled using an UPDATE statement applying the same formula.

6.5 Verification

Test insert with goals=2, assists=1, minutes_played=90 produced rating=6.0, confirming the formula: $(2 \times 2) + 1 + (90/90) = 6.0$.

7. RAG Pipeline (Track A)

7.1 Architecture

SportRAG implements a full retrieval-augmented generation pipeline without requiring any external LLM API key. The pipeline has three stages: embedding, retrieval, and answer generation.

Component	Technology	Detail
Embedding Model	sentence-transformers	all-MiniLM-L6-v2 (384 dimensions, runs locally)
Vector Store	pgvector	embedding column of type vector(384) on Documents table
Similarity Index	IVFFlat	Cosine distance index with 10 lists
Backend	FastAPI + Python	REST API on port 8000
Frontend	HTML/CSS/JS	Browser-based chat interface

7.2 Pipeline Flow

When a user submits a question, the /ask endpoint first checks whether the question matches a known factual pattern (top scorers, assists, highest scoring matches, ratings, team goals). If so, it queries the PostgreSQL database directly using optimised SQL and returns a precise answer with citations from the PlayerStats and Matches tables.

For open-ended questions, the pipeline embeds the question using the local sentence-transformers model and performs a cosine similarity search against the 1,001 embedded match report documents stored in pgvector. The top-5 most similar documents are retrieved and used to construct a grounded answer with source citations showing match name, date, score, and similarity score.

7.3 API Endpoints

Endpoint	Method	Description
GET /	GET	Health check
GET /status	GET	Embedding index progress
POST /index	POST	Embed all documents into pgvector (run once)
POST /ask	POST	RAG question answering with citations
GET /stats	GET	Live database counts and top scorers

8. Limitations and Future Work

8.1 Current Limitations

- The answer generation does not use a large language model — responses are template-based. Integration with an LLM API would produce more natural and flexible answers.
- The IVFFlat index requires documents to be embedded before similarity search works. A production deployment would automate this on data insertion via a trigger.
- The frontend must be served via a local HTTP server; it cannot be opened as a file due to browser CORS restrictions.
- Dataset is synthetically generated. Real match data would produce more meaningful retrieval results.

8.2 Future Work

- Integrate an LLM (e.g., Claude or GPT-4) for natural language answer generation
- Add player comparison queries and seasonal statistics
- Implement user authentication and query history
- Deploy the FastAPI backend to a cloud server for persistent 24/7 availability
- Add HNSW indexing (pgvector) for faster similarity search at scale

9. AI Usage Disclosure

In accordance with the Z2004 AI Tools Policy, the following AI tools were used during this project:

Tool	Used For	Changes Made
Claude (Anthropic)	Structuring the RAG pipeline (rag.py, main.py)	Reviewed, tested, and adapted to match our schema and Supabase connection
Claude (Anthropic)	Debugging the BEFORE INSERT trigger RETURN NEW bug	Understood root cause and applied fix independently in Supabase SQL Editor
Claude (Anthropic)	Frontend UI layout and styling (index.html)	Adapted API base URL, endpoint calls, and sidebar content to match our data

All SQL schema design, data generation logic, query writing, normalisation analysis, and index decisions were produced by the group. All AI-generated code was read, understood, and tested against the live database before submission.

10. Conclusion

SportRAG successfully demonstrates a complete Track A RAG Pipeline built on a normalised PostgreSQL database. The project covers all required technical components: a 3NF schema with five tables and over 12,000 rows of data, a ten-query SQL suite spanning aggregation, joins, subqueries, CTEs, and window functions, B-Tree and IVFFlat indexes with documented performance improvements, a working BEFORE INSERT trigger for automatic rating calculation, and a Python RAG pipeline that accepts natural-language questions and returns grounded answers with source citations.

The system runs entirely without external API keys, using a locally-executed sentence-transformers model for embeddings and a rule-based answer generator grounded in live database queries. The frontend provides an accessible chat interface that demonstrates the pipeline to non-technical users while the underlying SQL and indexing work demonstrates mastery of database management systems concepts.